

Solution Architecture

Date	
Team ID	XXXXXX
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Solution Architecture Diagram:

High-level view of EduGenie's Presentation Layer, Application (API + logic) Layer, AI integrations and Data Layer.

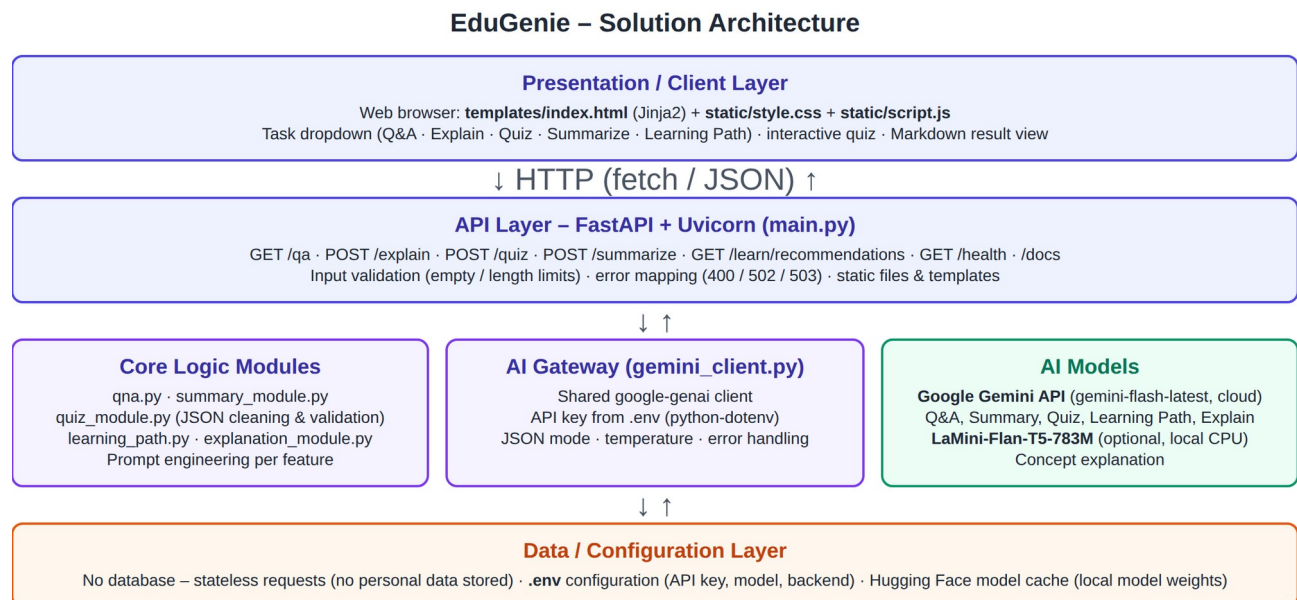


Figure: EduGenie solution architecture

Request Flow

1. The student selects a task and submits input in the browser (`index.html` + `script.js`).
2. `script.js` sends a `fetch()` request (GET/POST JSON) to the matching FastAPI endpoint.
3. `main.py` validates the input and calls the feature module (qna, explanation, quiz, summary, `learning_path`).
4. The module builds a task-specific prompt and calls Gemini through `gemini_client.py` (or the local LaMini-Flan-T5 model for explanations).
5. The response is cleaned/validated (quiz JSON) and returned as JSON; the browser renders it (formatted text or interactive quiz).

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Single-page UI: task dropdown, input area, options (quiz size, level), results with Markdown rendering, interactive quiz and score, status badge.	HTML5, Jinja2, CSS3, JavaScript (Fetch API)
API Layer (FastAPI)	Exposes REST endpoints, validates input, maps errors to HTTP status codes, serves templates/static files and API docs.	FastAPI, Uvicorn, Pydantic
Core Logic Modules	One module per feature with its own prompt: qna.py, explanation_module.py, quiz_module.py, summary_module.py, learning_path.py.	Python 3.10+
AI Gateway & Models	gemini_client.py manages the API key, model name, JSON mode and errors; Gemini generates content; LaMini-Flan-T5 explains concepts locally.	google-genai SDK, Gemini (gemini-flash-latest), transformers, torch
Data / Configuration	Stateless - no database. Configuration (API key, model, explanation backend) read from .env.	python-dotenv, .env file